



Московский
транспорт



Молоток

Кейс 05 · решение ReSense

Обнаружение посторонних объектов в тоннеле метро по данным 3D-лидара



О команде

- **Капитан:** <ФИО>, <специальность>
- **Кол-во участников:** 4 человека
- **Краткое описание:**
 - <как образовалась команда>
 - место учёбы: <место учёбы / работы>
- **Город и регион:** <город>

КОМАНДА «Молоток»

Краткое описание решения:

ROS 2-модуль: по потоку 3D-лидара 10 раз в секунду строит модель «нормального тоннеля» — полотно, рельсы, ось пути с кривизной по стенам, — проверяет габарит поезда 2,1 × 3,0 м и отвечает GO / CAUTION / STOP / FAULT с расстоянием до препятствия. Без обучения на объектах, без GPU, в Docker.

Уникальность решения:

Описываем среду, а не объекты: крепление лидара и ось пути калибруются по рельсам в каждом кадре, кривизна — по стенам и рядам колонн, поэтому коридор осмыслен и там, где рельсов уже не видно. Вместо молчания — «проверенно свободная» дистанция. Настройка и оценка — на препятствиях, вставленных трассировкой лучей в реальные кадры.

КОМАНДА «Молоток»



<Имя
Фамилия>

- Капитан · ROS 2, Docker, интеграция
- <@ник>
- <место учёбы / работы>

<Имя
Фамилия>

- Визуализация, демо, презентация
- <@ник>
- <место учёбы / работы>

<Имя
Фамилия>

- Компьютерное зрение: модель пути, трекинг
- <@ник>
- <место учёбы / работы>

<Имя
Фамилия>

- Данные, синтетика, метрики, тесты
- <@ник>
- <место учёбы / работы>

Краткая история команды:

Мы друзья и на конкурсы всегда выходим вчетвером: ROS 2 и интеграция, визуализация, компьютерное зрение, данные и тесты. Каждый день ездим на учёбу на метро — захотелось, чтобы поезд однажды сам видел, что у него впереди.

01

Почему вы выбрали именно эту задачу из предложенных на хакатоне?

Метро — наша ежедневная дорога, а беспилотный поезд — задача, где ошибка стоит дорого. Препятствий в данных почти нет, поэтому описываем «нормальный тоннель» и ловим всё, что в него не вписывается.

02

С какими основными сложностями или вызовами вы столкнулись и как их преодолели?

Препятствий в записях почти нет — «ставили» людей и ящики в реальные кадры трассировкой лучей лидара. Станции и стрелки давали ложные остановки — помогли ось по стенам, зона доверия и подтверждение 0,5 с. Записи разные (два топики, 120° и 360°, наклон крепления 3°) — узел сам находит вход и калибруется по рельсам. И всё это — параллельно с учёбой.

03

Техническая суть решения

- ROS 2 Humble-узел на Python (numpy / scipy / scikit-learn) и необязательные ядра на C++
- Вход — PointCloud2 любого из двух наборов топик / frame_id; выход — решение GO / CAUTION / STOP / FAULT, дистанция до препятствия и «проверенно свободная» дистанция, Detection3DArray, маркеры RViz, JSON-статус
- В каждом кадре: автокалибровка крепления → модель пути (полотно, рельсы, ось, кривизна по стенам) → габарит 2,1 × 3,0 м + низкие объекты на рельсах → кластеры → фильтры инфраструктуры → подтверждение 0,5 с
- 42–64 мс на кадр ($p95 \leq 78$ мс) на одном ядре; ядра на C++ сокращают время детектора на 38–57 %, выход тот же
- Все 13 759 реальных кадров: человек на пути найден в 58 из 61 кадров, 3,5 ложных события на км поездки
- Только CPU: GPU оценили и не берём — выигрыш ≤ 30 –45 мс на кадр, а контейнер с GPU не стартует без nvidia-container-toolkit

Маркетинговая суть решения

Применение: «взгляд вперёд» для беспилотного поезда (GoA3/4) и хозяйственных поездов; контроль габарита при обкатке линий

Развитие: скорость поезда от одометрии — узел её уже принимает (своя оценка по лидару: ошибка 0,07 м/с, пока опция); опора высоты по своду тоннеля — мелкие объекты дальше 100 м; «второе мнение» на реальных препятствиях

Внедрение: docker load → docker run → ros2 bag play; один файл параметров; стандартные сообщения ROS 2 — стыкуется с системой торможения без переделки

ПРОБЛЕМА И ПОДХОД



Проблема

Поезд без машиниста должен видеть путь: 100 м — хорошо, 200 — очень хорошо, 300 — отлично. Тормозной путь на 80 км/ч — 190–250 м. Тоннель однороден и почти всегда пуст: классов объектов для обучения нет.

Альтернативы

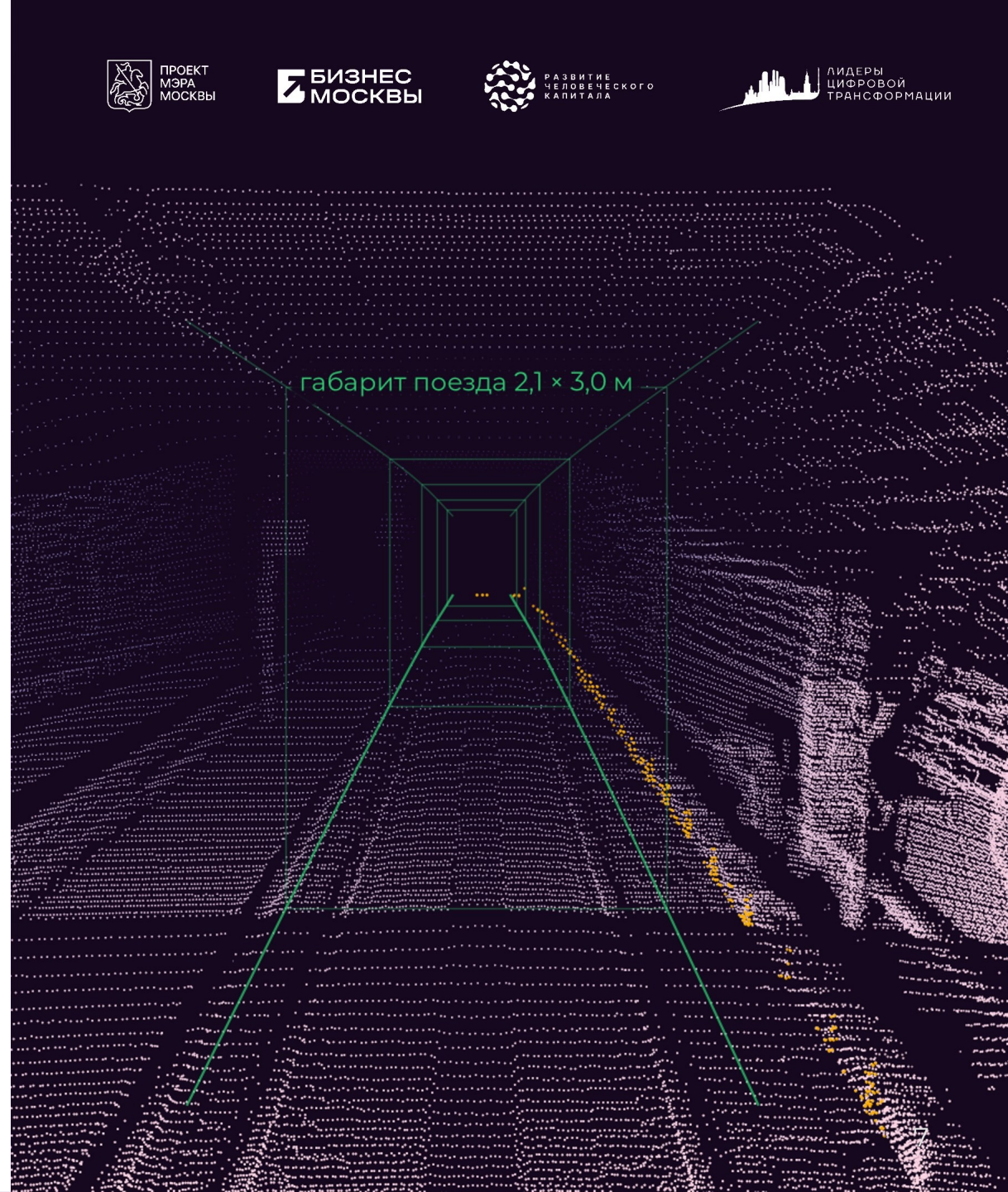
Обученный 3D-детектор (PointPillars, CenterPoint) — нужны тысячи размеченных препятствий, дальше 100 м — единицы процентов AP. Сравнение с картой — нужна сантиметровая локализация и повторный проезд.

Наше решение

Описать нормальный тоннель и сообщать всё, что попадает в габарит поезда 2,1 × 3,0 м. Модель пути строится заново в каждом кадре по рельсам, полотну и стенам — без карты и без классов объектов.

ДАННЫЕ И СЕНСОР

- **13 759 кадров** — 6 записей (2 488) и 20-минутная поездка (11 271 кадр, 13 км, 7 остановок, кривые до $R \approx 350$ м)
- **Hesai Pandar128**, 10 Гц, ~190 тыс. точек в кадре: стены видны на 150–200 м, полотно — до ~100 м
- **~210 м — предел отражений в тоннеле:** каждая запись обрывается на 209–210 м, дальше нет ни одной точки (паспорт: 200 м при 10 % отражения)
- Объект 0,5 м даёт ~7 точек на 100 м и 1–2 точки на 200 м
- Реальные препятствия: человек и предмет на рельсе в записи doubleT_obstacle; всё остальное — синтетика в реальных кадрах, каждое число помечено



АЛГОРИТМ



Автокалибровка

ориентация, крен и тангаж
лидара — по рельсам и
полотну: любое крепление

1

Габарит

коридор 2,1 × 3,0 м вдоль оси и
ступень низких объектов на
рельсах

3

Решение

подтверждение 0,5 с → GO /
CAUTION / STOP / FAULT и
свободная дистанция

5

Модель пути

полотно, головки рельсов, ось;
кривизна по стенам и
колоннам до 150–200 м

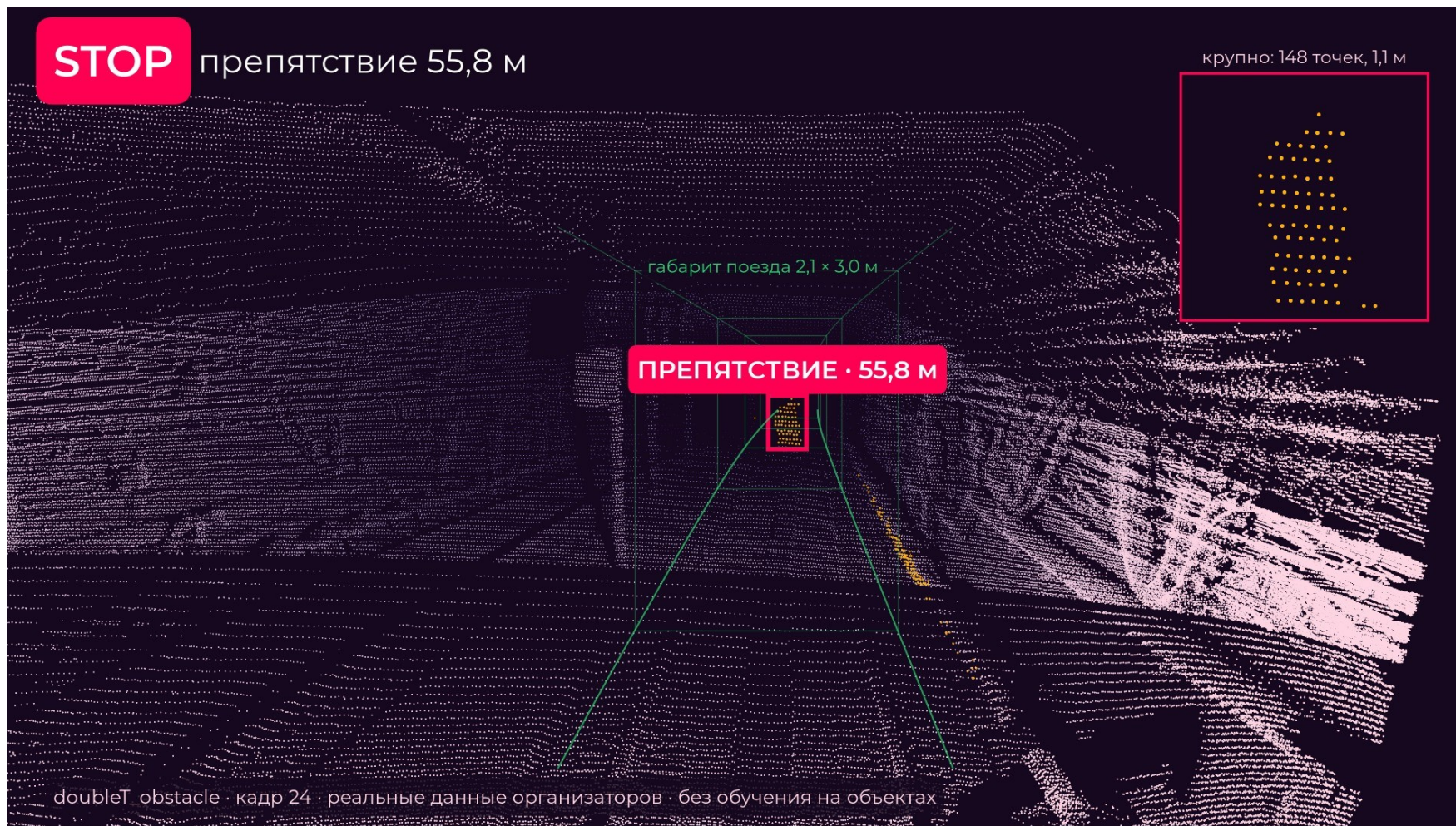
2

Кластеры

радиус растёт с дальностью;
фильтры инфраструктуры;
дальнее правило

4

ГЛАВНЫЙ КАДР



Реальная запись организаторов

Человек переходит путь на 55–57 м

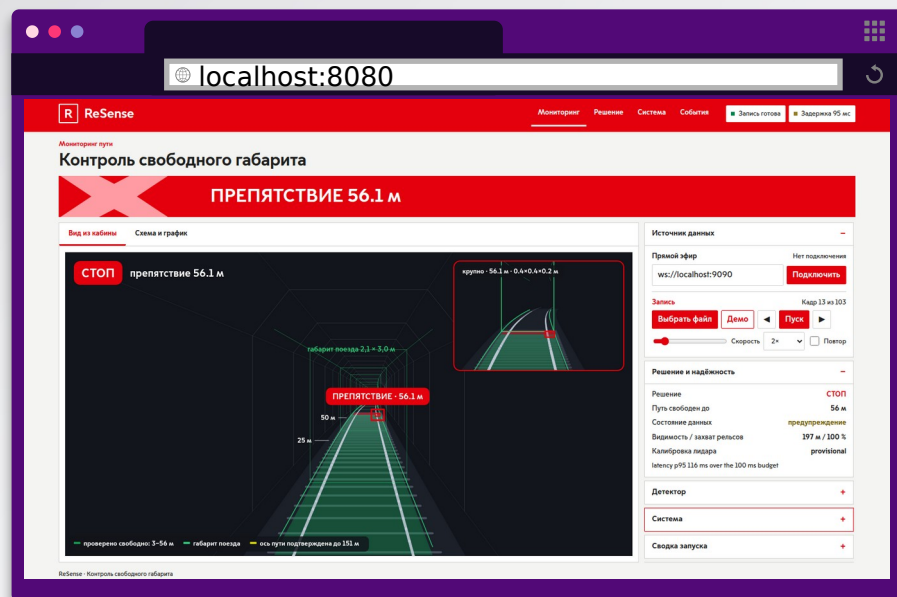
Найден в **58 из 61** кадров внутри габарита

Тревога через 0,3 с после входа в габарит

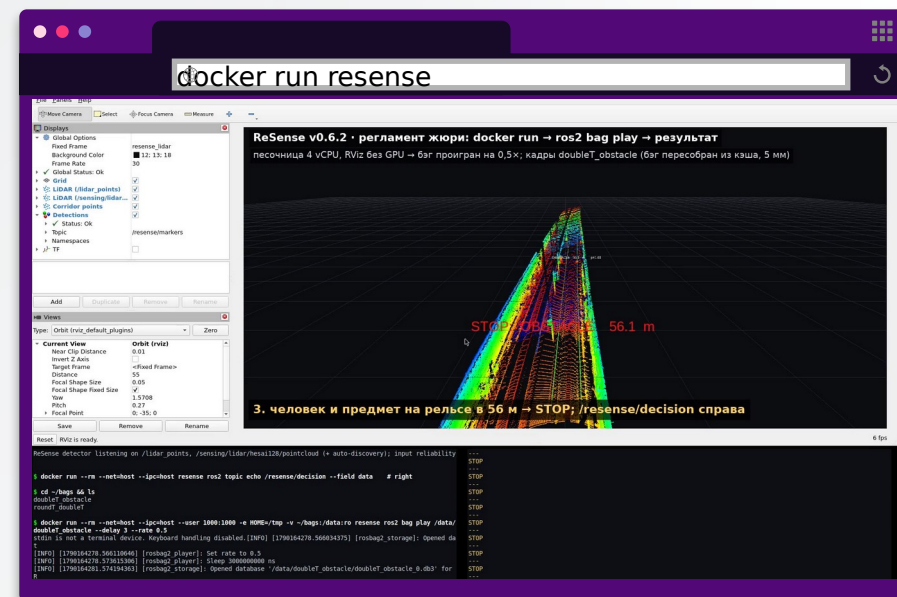
Ошибка дальности $\leq 0,23$ м

Предмет на рельсе — в **125 из 126** кадров после ухода человека

ДЕМОНСТРАЦИЯ



- Веб-дашборд «Контроль свободного габарита»: вид из кабины по статусу узла из Docker-прогона на doubleT_obstacle — STOP 56,1 м, путь свободен до 56 м; живой узел — через rosbridge



- Цепочка жюри в Docker: узел с RViz, bag play от обычного пользователя из другого контейнера, /resense/decision — STOP 56 м (видео 69 с)

docker load → docker run → ros2 bag play → /resense/decision

13 759

реальных кадров прогнаны целиком

58 из 61

кадров с человеком на пути — тревога

3,5 на км

ложных событий в поездке (45 за 13 км)

Реальная запись: человек на пути — 58 из 61 кадров, тревога через 0,3 с, ошибка $\leq 0,23$ м; предмет на рельсе — 125 из 126 кадров

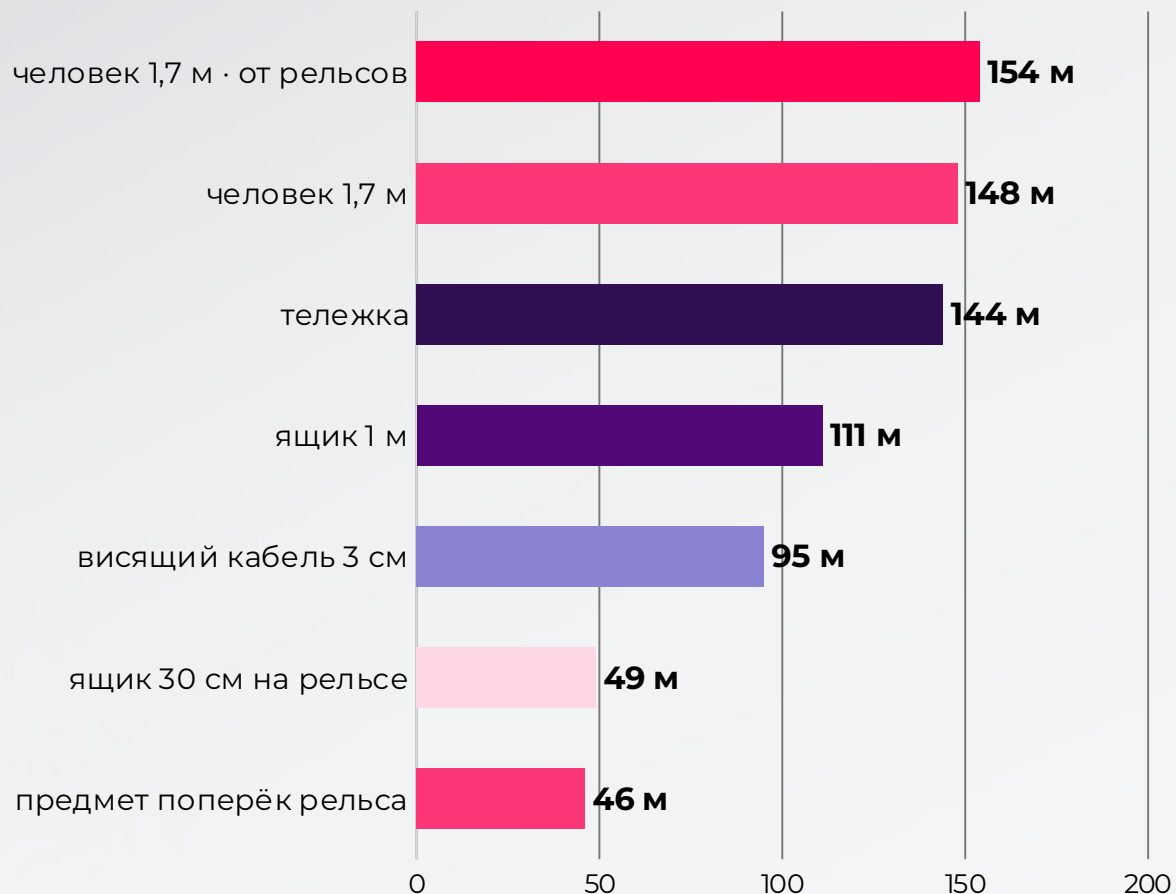
Ложные остановки (реальные): поездка 13 км — 45 событий, пять пустых записей — 13

Объекты организаторов [их синтетика]: ящик 2×2 м — с 98 м, доска — с 82 м, кубы 0,3 м — с 43–53 м; висящий 5 см — с 30 м; у края — лишь 2 и 6 кадров STOP

Человек на подходе [наша синтетика]: первое подтверждение 148 м (в 5 парах: 150 → 154 м от ближних рельсов), в каждой полосе 10 м — со 115 м; 167 м со скоростью поезда

Задержка 42–64 мс на кадр ($p95 \leq 78$ мс) на одном ядре; ядра на C++ сокращают время детектора на 38–57 %, выход тот же; без GPU

ДАЛЬНОСТЬ



первое подтверждение на прямой, медиана 6 подходов; «от рельсов» — объект поставлен по ближним рельсам, а не по дальней оси детектора (5 пар, прежняя постановка — 150 м); поезд 17–21 м/с без датчика скорости · синтетика в реальных кадрах поездки

- **~210 м — предел отражений в тоннеле**

дальше 210 м во всех 13 759 кадрах нет ни одной точки

- **Со скоростью поезда**

накопление 5 кадров: человек 167 м; своя скорость по лидару (ошибка 0,07 м/с) STOP раньше не даёт — опция

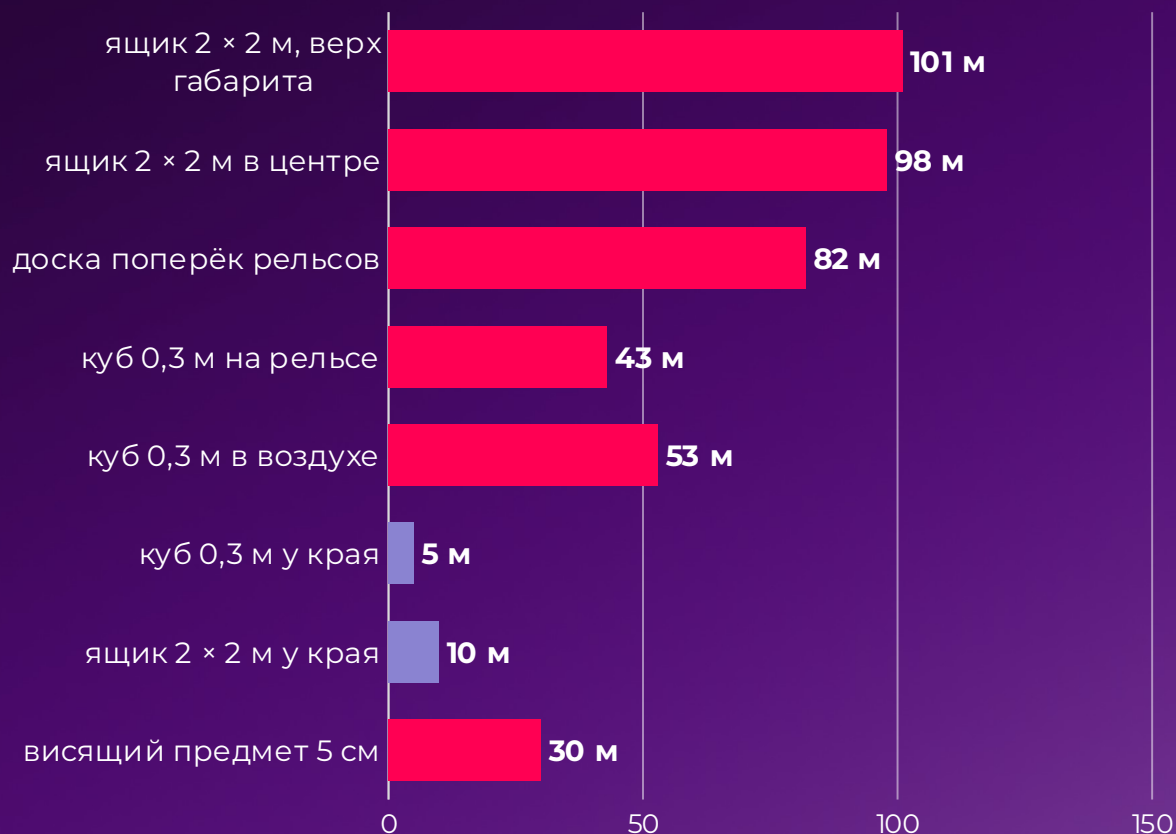
- **Устойчиво**

≥ 90 % кадров: человек с ~149 м, в каждом 10 м — со 115 м

- **В кривых**

6 из 7 подходов, с 58–86 м — предел видимости за стеной (R ≈ 350 м)

ОБЪЕКТЫ ОРГАНИЗАТОРОВ



первый STOP, м; сиреневым — без устойчивого STOP · cloud_with_fake_obj: 10 объектов, 1 510 кадров, поезд едет к ним 1,4–20 м/с, скорость узлу не дана · синтетика организаторов

• STOP — 8 из 8 объектов в габарите

ящик 2 × 2 м в центре — с 98 м, с первого появления; доска поперёк рельсов — с 82 м; кубы 0,3 м — с 43–53 м

• Мелкое вдали — поздно

на 60–115 м от куба 0,3 м 2–4 точки в кадре, кластеру нужно 5; ящик у верха габарита — STOP в 51 из 124 кадров — непрерывно с 101 м, дальше CAUTION или ничего

• Ограничения — говорим честно

краевые объекты дают лишь 2 и 6 STOP кадров; висящий предмет 5 см: STOP с 30 м; объекты ставили от оси лидача ($-0,24^\circ$ к рельсам), мы строим габарит по рельсам. Ложных STOP: 6 кадров у ящика снаружи, 3 — вне объектов

01

Любое крепление

- 8 ориентаций, крен и тангаж по рельсам и полотну; проверено на повернутых реальных кадрах

02

Монитор исправности

- ошибки входа → FAULT; задержка → предупреждение без смены решения

03

Любой вход

- оба набора топик / frame_id, поиск топика, перезапуск на новой записи

04

Воспроизводимо

- 587 тестов на путях nupru и C++ (выход совпадает бит в бит); CI собирает образ и проигрывает бэги через узел; цепочка организаторов прогнана в Docker на реальных записях

Полотно полно предметов. «Всё выше полотна» — 1 482 ложных события за 20 мин; порог « ≥ 3 см над головкой рельса + 0,5 с» — единицы

Предмет на рельсе делился на две ступени. Верх на 0,10–0,15 м — между ступенью низких объектов и габаритом; кластеризуем его целиком: 2 → 125 из 126 кадров

Станции и стрелки. Край платформы и конструкции у торца — главный источник ложных остановок; без рельсов в ближней зоне дальше 40 м — только предупреждение

Дальше 100 м — 1–5 точек на объект. Высокие объекты — по дальнему правилу до ~150 м; мелкие (< 0,6 м) — ближе: кубы 0,3 м организаторов — только с 43–53 м (на 60–115 м — 2–4 точки)

Объекты организаторов у края и тонкие. Ящик 2 × 2 м у края — STOP лишь с 10 м (6 кадров): их объекты стоят от оси лидача (–0,24° к рельсам), наш габарит — от рельсов; висящий предмет 5 см — STOP только с 30 м

01

Что получилось

- ROS 2-модуль в Docker; все 13 759 реальных кадров: человек и предмет на рельсе найдены, 3,5 ложных события на км; ящик организаторов — с 98 м; 587 тестов и CI

02

Что дальше

- мелкие объекты раньше: короткие сигнатуры инфраструктуры, опора высоты по своду тоннеля; скорость от одометрии (своя по лидару — опция); «второе мнение» на реальных препятствиях; репетиция офлайн-запуска

03

Внедрение

- docker load → run → ros2 bag play; один файл параметров; стандартные сообщения ROS 2 — стыкуется с системой торможения